Reflexión Act 2.3: Listas doblemente ligadas en el análisis de una bitácora

Actividad Integral de estructuras de datos lineales — TC1031 — Tec de Monterrey

Gerardo Alberto Garay A01564643

1. El problema

La actividad pide leer una bitácora de N registros (mes, día, hora, IP y mensaje), ordenarla por IP, recibir del usuario una IP inicial y una final, desplegar todos los registros dentro de ese rango y guardar el resultado ordenado en un archivo. Es decir, hay tres operaciones que dominan el costo: ordenar una sola vez, buscar los límites del rango y recorrer los registros que caen dentro de él.

2. ¿Qué es una lista doblemente ligada?

Es una estructura lineal en la que cada nodo guarda su dato y dos apuntadores: uno al nodo siguiente y otro al anterior. Mantener la referencia a la cabeza y a la cola permite insertar o eliminar en ambos extremos en O(1) y recorrer en las dos direcciones. Si ya se tiene un apuntador al nodo, insertar o borrar justo ahí también es O(1), porque no hace falta recorrer la lista para encontrar al nodo previo, como sí pasa en una lista simplemente ligada.

3. Complejidad de las operaciones solicitadas

Operación	Lista doblemente ligada	Vector (arreglo dinámico)
Leer N registros del archivo	O(N) (inserción al final en O(1) con cola)	O(N) amortizado (push_back)
Ordenar por IP	O(N log N) con merge sort; quick sort y heap sort pierden eficiencia sin acceso aleatorio	O(N log N) con merge sort; también permite quick sort in situ
Buscar la IP inicial y la final	O(N): no hay acceso directo al índice medio, la búsqueda binaria no aplica	O(log N) con búsqueda binaria (lower/upper bound)
Desplegar los K registros del rango	O(K) una vez ubicado el inicio	O(K)
Total por consulta	O(N + K)	O(log N + K)
Guardar el resultado en archivo	O(N)	O(N)
Memoria extra	Dos apuntadores por nodo: O(N) con constante más alta y peor localidad de caché	Memoria contigua, O(N)

El punto clave: la ordenación cuesta lo mismo en ambas estructuras, pero la **búsqueda** no. Como la actividad consiste en consultar rangos, lo que importa es poder saltar directo a los límites, y eso lo da el acceso aleatorio del vector.

4. Ventajas y desventajas en esta situación

Ventajas de la lista doblemente ligada

- Insertar y eliminar en cualquier posición conocida es O(1), sin desplazar elementos.
- El recorrido en ambas direcciones permite mostrar el rango de menor a mayor IP o al revés sin trabajo extra.
- Una vez ubicado el nodo de la IP inicial, avanzar hasta la final es natural: solo se siguen los apuntadores.
- No requiere memoria contigua ni reasignaciones cuando el contenedor crece.
- Merge sort sobre una lista enlazada se puede hacer sin copiar elementos, solo reenlazando nodos.

Desventajas

- No permite búsqueda binaria**.** Encontrar un límite cuesta O(N) en lugar de O(log N), y hay que hacerlo dos veces por consulta.
- Mayor consumo de memoria por nodo (dos apuntadores adicionales) y peor localidad de caché, lo que hace lentos los recorridos en la práctica.
- Mayor complejidad de implementación: es más fácil cometer errores con los apuntadores, y cada nodo debe liberarse explícitamente.
- Las ventajas de inserción y borrado en O(1) casi no se usan aquí, porque la bitácora se lee una vez y después solo se consulta.

5. Conclusión: ¿es la estructura más adecuada?

Para esta situación problema, no es la más adecuada. La bitácora es un conjunto de datos que se carga una vez, se ordena una vez y después se consulta muchas veces por rangos. Es un patrón de “muchas lecturas, casi ninguna modificación”, y ahí el arreglo ordenado con búsqueda binaria gana: cada consulta cuesta O(log N + K) frente a O(N + K) de la lista, con menos memoria y mejor uso de caché. Con una bitácora de decenas de miles de registros, la diferencia entre unas 16 comparaciones y decenas de miles por cada límite es notable.

La lista doblemente ligada sería una buena opción si el problema cambiara: por ejemplo, si llegaran registros nuevos de forma continua y hubiera que insertarlos en su posición ordenada, si se eliminaran registros con frecuencia, o si se necesitara navegar hacia adelante y hacia atrás desde un registro encontrado (paginación). Incluso entonces, un árbol binario de búsqueda balanceado daría inserción y búsqueda en O(log N).

En mi implementación usé un vector de apuntadores a *Entry*, merge sort con una función getter que extrae la IP, y dos búsquedas binarias (primer elemento mayor o igual que la IP inicial, y primer elemento mayor que la IP final) para delimitar el rango. Aprendí que la elección de la estructura se decide por las operaciones que dominan el problema, no por cuál estructura es “más avanzada”.

6. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4a ed.). MIT Press.
- Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4a ed.). Pearson.
- Stroustrup, B. (2013). *The C++ Programming Language* (4a ed.). Addison-Wesley.